AI Structural-Break Detector

for Statistical Arbitrage

Complete Project Overview and System Design

Postgraduate group project - Track A (Tech Team)

Current working specification: finance, econometrics, AI/ML, data, engineering, evaluation, and UI

Design principle

The application is an AI-in-Finance research system first. Financial econometrics creates and validates relative-value opportunities; supervised ML ranks those opportunities; language AI investigates structural-break evidence; deterministic controls govern the final research decision.

Scope boundary

The deliverable is a research and decision-support platform, not an execution engine. It does not place live orders, claim deployable alpha, or treat LLM output as authoritative market data.

Contents

1. System definition and final product scope
2. End-to-end architecture
3. Financial market scope and trading research design
4. Data architecture and source strategy
5. Econometric relationship engine
6. Divergence-event dataset and target construction
7. Feature engineering
8. Supervised ML layer
9. Regime and non-stationarity layer
10. NLP and point-in-time event intelligence
11. Multi-agent structural-break investigation
12. Decision controller and risk gates
13. Backtesting and evaluation framework
14. Complete user interface specification
15. Exact technical stack and engineering architecture
16. Data model, repository structure, and system jobs
17. Reproducibility, testing, governance, and monitoring
18. Prototype versus real deployment
19. Build sequence and acceptance criteria
Appendix A. Working configuration defaults
Appendix B. App-specific concept glossary

Document use

This document is the top-to-bottom system specification for the project. It deliberately excludes idea-generation material, promotional pitch language, concept analogies, and mathematical derivations. Parameters shown as defaults are configuration choices to be validated on training/validation data before the test period is opened.

1. System definition and final product scope

1.1 Product definition

The application is an AI-enhanced relative-value research platform for equity pairs. It monitors a curated universe of economically related securities, detects statistically unusual divergences, estimates the probability that each divergence will mean-revert, investigates whether public company information supports a structural-break explanation, and returns a bounded research status: CANDIDATE, REVIEW, NO TRADE, or ABSTAIN.

The central design requirement is separation of responsibilities. The statistical layer identifies unusual relative-value states; the ML layer produces a calibrated prediction from structured features; the language-AI layer extracts and challenges event evidence; and a deterministic controller applies risk gates. No single layer is allowed to silently replace another.

1.2 Primary users and use case

User	How the app is used
Quant / systematic researcher	Screens statistically valid pair dislocations and ranks them by estimated mean-reversion quality.
Multi-strategy research desk	Uses event evidence and structural-break diagnostics to decide which quantitative signals require human review.
Model-risk / academic user	Studies when historical pair relationships fail and whether AI filters reduce false opportunities.

1.3 Working project scope

Scope item	Working specification
Asset class	US listed large-cap equities only.
Pair universe	20-40 manually curated same-sector / same-sub-industry pairs with clear economic similarity and adequate liquidity.
Frequency	Daily data. No intraday execution or high-frequency claims.
Historical period	2016-2025 raw data; earliest usable events occur after the rolling formation window.
Core model	XGBoost binary classifier with probability calibration and SHAP explanations.
Text evidence	SEC EDGAR filings as the canonical historical source; historical news is an enrichment layer where timestamp quality is acceptable.
Agent layer	Specialised LLM agents with structured outputs, source provenance, and explicit abstention.
UI	Multi-page Streamlit research dashboard with Plotly visualisations and cached historical results.
Execution	No live brokerage integration; trade logic is simulated only for backtesting and research.

1.4 Explicit non-goals

- No prediction of raw stock prices or market direction.
- No search across every possible stock pair; the universe is economically pre-filtered.
- No high-frequency trading, order-book modelling, or latency-sensitive execution.
- No autonomous live trading. The strongest system output is a research candidate, not an order.
- No claim that a backtest proves deployable profitability.
- No LLM-generated prices, probabilities, source facts, or unsupported catalysts.

- No deep-learning price model unless the core supervised and agent layers are complete and validated.

2. End-to-end architecture

2.1 System pipeline

1. Market and document ingestion

Daily adjusted prices, volume, sector metadata, market/regime series, SEC filings, optional timestamped news.

↓

2. Pair relationship engine

Rolling relationship estimation, pair-validity diagnostics, spread state, z-score, half-life and stability measures.

↓

3. Divergence event generator

Creates one event when a valid pair crosses the configured divergence threshold; suppresses overlapping duplicates.

↓

4. Feature snapshot

Freezes only information available at the event timestamp and stores the full model feature vector.

↓

5. ML inference

XGBoost predicts a calibrated probability of successful mean reversion; SHAP identifies the main feature contributions.

↓

6. Structural-break investigation

Retrieves admissible public evidence and runs structured catalyst, mean-reversion, structural-break, and validation agents.

↓

7. Deterministic decision controller

Combines ML score, evidence risk, data quality, liquidity, and policy rules into CANDIDATE / REVIEW / NO TRADE / ABSTAIN.

↓

8. Research UI and evaluation

Scanner, pair detail, evidence timeline, strategy lab, model/data monitoring, and model-card pages.

2.2 Offline versus interactive workloads

Workload	When it runs	Output
Historical ingestion	Scheduled or manually triggered before model runs	Versioned price, metadata, regime, and document stores.
Pair estimation and event creation	Batch job	Point-in-time pair snapshots and divergence-event table.
Feature construction and labels	Batch job	Leakage-safe supervised-learning dataset.
Model training and calibration	Experiment job	Versioned model artefact, metrics, calibration object, SHAP reference data.
Historical agent evaluation	Batch or cached job	Structured event extractions, agent findings, evidence links, veto/abstention outcomes.
Dashboard inference	Interactive	Loads cached quantitative results; optionally runs agent investigation for a

Workload	When it runs	Output
		selected event and caches the result.

Architecture rule
Every stage writes a structured artefact that can be inspected independently. The user should be able to trace a final decision backward from the controller to the agent findings, ML prediction, feature snapshot, pair diagnostics, and raw source timestamps.

3. Financial market scope and trading research design

3.1 Relative-value research object

The app operates on equity pairs whose businesses and risk exposures are sufficiently similar that a stable historical relationship is economically plausible. Each pair is treated as a research object with metadata, validity state, current spread state, model score, event evidence, and a lifecycle status.

3.2 Pair-universe construction

Filter	Implementation in the app
Economic similarity	Manual curation by sector, sub-industry, business model, and geographic exposure.
Liquidity	Minimum average daily dollar volume and price-history completeness.
Listing history	Prefer long-lived securities with continuous history across the project sample.
Correlation pre-filter	Used only as a screening aid; it is not treated as proof of a stable spread relationship.
Econometric validity	Only pairs that pass the rolling relationship test are eligible to generate events.
Pair lifecycle	ACTIVE, WATCH, or RETIRED based on repeated validity and stability checks.

3.3 Research trade representation

- A positive or negative divergence determines which leg is relatively rich and which is relatively cheap according to the estimated relationship.
- The backtester represents the research idea as a long/short pair with hedge-ratio-based sizing and normalised gross exposure.
- An event closes when the spread reaches the configured convergence zone, reaches the adverse stop, or reaches the maximum holding horizon.
- Each event is treated as a distinct research opportunity. New overlapping events for the same pair are suppressed until the previous event resolves.
- Transaction costs and short-side carrying costs are included as configurable research assumptions, then varied in sensitivity analysis.

3.4 Finance outputs surfaced to users

Output	Where it appears
Current spread and divergence level	Scanner and pair detail.
Long / short research direction	Pair detail and simulated trade panel.
Estimated convergence horizon / half-life	Pair diagnostics.

Output	Where it appears
Liquidity and implementation flags	Scanner, pair detail, and controller.
Gross and net simulated event return	Strategy Lab.
Sharpe, maximum drawdown, turnover, tail loss, win rate	Strategy Lab and model comparison tables.
Transaction-cost sensitivity	Strategy Lab scenario panel.

4. Data architecture and source strategy

4.1 Core source hierarchy

Data domain	Prototype source	Use in the system	Main control
Adjusted daily OHLCV	Yahoo Finance via yfinance	Pair estimation, returns, spread state, volume/liquidity features, backtest.	Cache raw downloads; retain corporate-action-adjusted series; freeze dataset version.
Company / ticker metadata	Curated pair file plus SEC ticker-CIK mapping	Sector tags, company identity, filing retrieval.	Manual review of pair metadata and ticker changes.
SEC filings	SEC EDGAR official public filings	Canonical historical evidence for earnings, guidance, M&A, management, litigation, and material events.	Store filing timestamp and accession identifier; do not use documents published after the event.
Historical news	GDELT or another timestamped public corpus where feasible	Optional catalyst enrichment and demo evidence.	Treat as optional; exclude from rigorous historical evaluation if timestamp/coverage cannot be defended.
Market / regime series	Broad-market ETF, sector ETFs, VIX or equivalent public series	Market return, sector return, volatility, stress/regime features.	Align trading dates and use only prior/current observations.

4.2 Canonical historical evidence policy

For rigorous historical agent evaluation, SEC filings are the canonical evidence source because publication dates and filing identifiers are reproducible. News can enrich current/demo investigations, but it should not determine headline historical results unless the team can prove point-in-time coverage and timestamp fidelity.

4.3 Point-in-time information policy

Allowed at event time	Not allowed
Prices, volume, regime values, filings and news published at or before the event timestamp.	Any future price movement except for constructing the outcome label.
Pair parameters estimated from the historical formation window.	Hedge ratios or spread statistics fitted using future observations.
Known corporate-action adjustments reflected in the selected historical price source.	Post-event analyst summaries that explain the event with hindsight.
Previously released filings and event extractions.	Future earnings, future filings, or a later restatement used as if known at entry.

4.4 Raw, processed, and model-ready data zones

Zone	Contents	Storage
Raw	Downloaded prices, SEC HTML/text, news payloads, market series, source metadata.	Parquet / compressed text files with immutable timestamped snapshots.
Processed	Clean prices, aligned calendars, pair metadata, document text, market features.	Parquet tables plus DuckDB views.

Zone	Contents	Storage
Analytics	Pair snapshots, divergence events, feature rows, labels, predictions, agent outputs, decisions, backtest trades.	DuckDB tables keyed by stable IDs.
Artefacts	Model file, calibrator, feature schema, SHAP background sample, experiment metrics, model card.	Versioned artefact directory and MLflow run metadata.

5. Econometric relationship engine

5.1 Role of econometrics in the application

Econometrics is supporting infrastructure. Its job is to define a defensible relative-value relationship, determine whether a pair is currently eligible for the AI pipeline, and generate interpretable diagnostics. It does not make the final trade decision and is not the main research contribution.

5.2 Core relationship method

Component	Working design
Price transformation	Use log adjusted closing prices for the relationship model.
Relationship model	Two-variable Engle-Granger cointegration workflow.
Formation window	Rolling 504 trading days.
Re-estimation cadence	Monthly for historical reconstruction and current scanner refresh; all estimates use information available at that date.
Eligibility threshold	Cointegration significance threshold fixed from training/validation policy; 5% is the default starting rule.
Hedge ratio	Estimated inside each rolling formation window and frozen for the corresponding signal period.
Spread	Residual-based relative-value state generated from the fitted pair relationship.
Divergence standardisation	Spread converted into a rolling standardised score used by the event generator and UI.
Half-life	Estimated mean-reversion speed used as a feature and diagnostic; not treated as a guaranteed holding period.

5.3 Relationship-stability diagnostics

- Rolling cointegration test statistic and p-value history.
- Rolling correlation over short and medium windows.
- Hedge-ratio drift relative to the previous estimation windows.
- Spread volatility and recent change in spread volatility.
- Frequency and recency of equilibrium crossings.
- Repeated invalidity flag: a pair moves from ACTIVE to WATCH or RETIRED after repeated failure of the relationship criteria.

5.4 Pair lifecycle logic

Status	Meaning	Event generation
ACTIVE	Current rolling relationship is valid and stability checks are within limits.	Enabled.
WATCH	Recent deterioration or one failed re-estimation window.	Disabled or routed only to REVIEW, depending on configuration.

Status	Meaning	Event generation
RETIRED	Repeated invalidity, major corporate transformation, delisting, or unusable liquidity/history.	Disabled.

6. Divergence-event dataset and target construction

6.1 Unit of observation

The supervised dataset is event-based. One row represents one historical moment when an eligible pair crosses the pre-defined divergence threshold. This is preferable to treating every trading day as an independent observation because the model is meant to score candidate dislocations, not ordinary days.

6.2 Event-generation policy

Rule	Default working specification
Entry condition	Absolute spread z-score reaches or exceeds 2.0 while the pair is ACTIVE.
Event timestamp	End-of-day signal timestamp; all features are frozen using information available by that timestamp.
Duplicate control	No new event for the same pair while a prior event remains unresolved or inside its maximum label horizon.
Convergence success	Spread returns to within 0.5 standard deviations of equilibrium before the failure condition and within 10 trading days.
Failure condition	Spread reaches the 3.0 standard-deviation adverse boundary before successful convergence, or fails to converge by the 10-day horizon.
Direction	Stored explicitly so features can distinguish positive and negative divergence states without leaking the future outcome.

6.3 Chronological partitioning

Partition	Working dates	Use
Training	Usable events from 2018-2021	Fit model and feature transformations; hyperparameter search occurs only through training/validation workflow.
Validation	2022-2023	Select thresholds, calibrate probabilities, compare features, set decision policy.
Final test	2024-2025	Opened once; used for headline out-of-sample results.
Embargo	At least the label horizon around partition boundaries	Prevents a training event label from using price observations inside the next partition.

Leakage rule

Future data is permitted only for constructing the outcome label. Every predictor, pair parameter, document, regime value, and LLM evidence item must exist in the historical information set at the event timestamp.

7. Feature engineering

7.1 Feature families

Feature family	Core variables in the model
Spread state	Current z-score magnitude and sign; recent spread change / velocity; recent spread volatility; distance from equilibrium; number and recency of recent equilibrium crossings.
Relationship quality	Cointegration statistic and p-value; estimated half-life; short- and medium-window correlation; hedge-ratio drift; recent relationship-validity history.
Market / regime	Broad-market returns; sector ETF returns; VIX or volatility proxy; broad realised volatility; optional HMM regime state.
Security state	Relative momentum between the two legs; abnormal volume; relative realised volatility; one-leg-versus-sector abnormal move; recent price gaps.
Liquidity / implementation	Average dollar volume; volume imbalance; price level; turnover proxy; simple daily-data trading-cost proxy; short-side implementation flag if available.
Event / NLP	Structured catalyst type; affected leg; materiality; one-sided-event flag; source count; document recency; confidence. Used only when point-in-time evidence is defensible.

7.2 Feature controls

- All rolling features are computed with trailing windows only.
- Feature definitions are versioned in a single schema shared by training and inference.
- Missing values are explicit; the system never silently substitutes future information.
- Highly redundant or unstable features can be removed using training/validation evidence, not test-period performance.
- Point-in-time fundamental valuation variables are excluded from the core model unless a reliable historical source is available.

7.3 Why the core model remains tabular

The event dataset is structured and likely moderate in size. The system therefore uses engineered financial, econometric, market, and event features rather than an end-to-end deep neural network. This keeps the predictive component trainable on course-scale infrastructure, supports SHAP explanations, and makes ablation testing straightforward.

8. Supervised ML layer

8.1 Prediction task

The ML layer is a binary classifier. For each divergence event it estimates the calibrated probability that the spread will satisfy the pre-defined mean-reversion success rule. The model does not predict the exact future stock price, the exact trade return, or whether a live trade should be executed.

8.2 Model stack

Model	Role
Logistic regression	Transparent benchmark for incremental value testing and probability sanity checks.
XGBoost	Primary classifier. Handles non-linear interactions across tabular event features and supports strong post-hoc explanation.
Probability calibrator	Maps raw model scores into more reliable probabilities using validation data. Default: sigmoid/Platt calibration because the event sample may be too small for flexible non-parametric

Model	Role
	calibration.
SHAP explainer	Provides local feature attribution for each event and global feature-importance summaries.

8.3 Training policy

- Chronological training and validation only; no random train/test split for headline results.
- Hyperparameters are selected on training/validation data, never on the final test period.
- Class imbalance is measured explicitly. Class weights or threshold tuning may be used if required, but resampling choices must preserve time structure.
- The classifier threshold for candidate selection is a policy parameter chosen on validation data; it is not automatically fixed at 50%.
- All preprocessing and model artefacts are saved together so inference applies the exact same transformations as training.

8.4 Model outputs shown in the app

Output	Interpretation in the app
Calibrated reversion probability	Primary numeric ML score for the current event.
Probability bucket	Low / medium / high model-confidence grouping used in scanner and evaluation.
Top SHAP drivers	Features that pushed the current event toward or away from predicted success.
Calibration context	Shows whether historical events with similar predicted probabilities actually reverted at similar rates.
Model version	Identifies the exact trained artefact used for the displayed result.

9. Regime and non-stationarity layer

9.1 Core regime features

The core version uses observable market-state features rather than making regime modelling a separate research objective. Broad-market return, sector return, VIX/volatility, and realised volatility enter the ML feature set because the behaviour of mean reversion may differ materially in stressed and calm markets.

9.2 Optional HMM regime state

A Hidden Markov Model may be added as a compact regime feature if it can be implemented point-in-time without destabilising the project. It would classify recent market conditions into a small number of latent states such as calm, normal, and stressed. The state is then treated as one input to XGBoost and a segmentation variable in evaluation; it does not replace the pair model.

9.3 Non-stationarity controls

- Rolling pair re-estimation rather than one relationship fitted over the full sample.
- Pair retirement when the historical relationship repeatedly fails validation.
- Performance reporting by calendar period and market regime.
- Model drift monitoring for feature distributions and reversion success rates.
- No assumption that a pair or model remains valid indefinitely.

10. NLP and point-in-time event intelligence

10.1 Document ingestion

When a divergence event is investigated, the text subsystem retrieves public evidence associated with both companies. For historical evaluation, the default evidence window is recent filings available before the event timestamp, with emphasis on 8-K filings and relevant sections of 10-Q/10-K filings. Optional historical news is added only when its publication time can be defended.

10.2 Document-processing pipeline

Stage	Function
Retrieve	Query company identifiers and fetch admissible documents dated at or before the event timestamp.
Clean	Remove HTML boilerplate, navigation text, duplicated exhibits where practical, and preserve document metadata.
Chunk	Split long documents into semantically coherent sections with source identifiers and page/section metadata when available.
Retrieve relevant evidence	Search chunks using company names, event keywords, and semantic similarity so the LLM sees a bounded evidence set rather than entire filing histories.
Structured extraction	Convert cited evidence into a fixed event schema validated by Pydantic.
Persist	Store extracted fields, citations/source IDs, prompt/model metadata, timestamps, and confidence.

10.3 Structured event schema

Field	Allowed / expected content
event_type	EARNINGS, GUIDANCE, M&A, REGULATORY, LITIGATION, MANAGEMENT, PRODUCT, FINANCING, MACRO, SECTOR, NONE, OTHER.
affected_leg	A, B, BOTH, or UNKNOWN.
direction	Positive, negative, mixed, neutral, or unknown relative to the affected security.
materiality	LOW, MEDIUM, HIGH, or UNKNOWN.
relationship_relevance	Whether the evidence plausibly changes the relative economic relationship between the pair.
confidence	Agent confidence in the extraction itself; never a trading probability.
evidence	Short evidence snippets or paraphrases tied to stored source identifiers.
source_timestamp	Publication or filing timestamp used to enforce point-in-time validity.

10.4 LLM evidence rules

- The model may classify and summarise retrieved evidence but may not invent an event that is absent from the sources.
- Every material claim must carry source provenance in the structured output.
- If the evidence set is empty, contradictory, stale, or inaccessible, the correct output is uncertainty or ABSTAIN rather than a fabricated conclusion.
- The LLM never supplies the market price, econometric statistics, or XGBoost probability; these values are injected from deterministic system state.

11. Multi-agent structural-break investigation

11.1 Agent roles

Agent	Mandate	Permitted evidence / outputs
Catalyst Extraction Agent	Identify and classify material public events around the divergence.	Retrieved source chunks -> structured event schema with provenance.
Mean-Reversion Advocate	Build the strongest evidence-based case that the divergence is temporary and the historical relationship may remain valid.	Sector-wide move, stable pair diagnostics, absence of one-sided material evidence, analogous prior events.
Structural-Break Advocate	Build the strongest evidence-based case that new information justifies a persistent repricing.	One-sided earnings/guidance, M&A, regulation, litigation, product or management events, major business-model change.
Risk / Validation Agent	Check whether the two arguments use valid sources, respect timestamps, acknowledge missing data, and remain consistent with deterministic inputs.	Structured validation flags; source completeness; contradiction detection; abstention recommendation.

11.2 Orchestration pattern

Retrieve admissible evidence

Only public evidence available by the event timestamp.

↓

Catalyst extraction

Produces the shared structured event state.

↓

Parallel adversarial analysis

Mean-Reversion Advocate and Structural-Break Advocate see the same evidence and quantitative state.

↓

Risk / validation review

Checks citations, contradictions, missing information, and whether either agent exceeded its mandate.

↓

Structured break-risk output

LOW / MEDIUM / HIGH / UNKNOWN plus validated evidence and uncertainty flags.

11.3 Agent state and controls

- The orchestration uses a typed shared state rather than free-form conversation history.
- Agents receive only the fields required for their task plus retrieved evidence; they do not query unrestricted web sources during historical backtests.
- Outputs are JSON-schema constrained and validated before downstream use.
- Tool timeouts, retry limits, and maximum reasoning steps prevent the demo from hanging.
- Agent outputs are cached by event ID, source-document version, prompt version, and model configuration.
- The controller can ignore an agent result that fails validation and move to ABSTAIN.

Governance refinement

The final research status is not delegated to a free-form Decision Agent. LLM agents produce evidence and a structured break-risk assessment; a deterministic controller applies the final decision policy. This reduces hallucination risk and makes historical evaluation reproducible.

12. Decision controller and risk gates

12.1 Inputs to the controller

Input class	Examples
Pair validity	ACTIVE/WATCH/RETIRED state, current cointegration validity, hedge-ratio stability.
ML output	Calibrated reversion probability, model version, missing-feature flags.
Agent output	Break risk, catalyst materiality, affected leg, evidence quality, uncertainty.
Market state	Stress/regime indicators and unusual volatility.
Implementation	Liquidity checks and cost/short-side flags.
Data quality	Stale inputs, missing price history, unavailable filings/news, failed agent run.

12.2 Default decision policy

Status	Default conditions
CANDIDATE	Pair is ACTIVE; data-quality and liquidity gates pass; calibrated reversion probability is high (default starting threshold 0.65); structural-break risk is LOW; no high-materiality one-sided event.
REVIEW	Intermediate model score, MEDIUM break risk, unusual regime, conflicting evidence, or a soft implementation warning. Human review is explicitly required.
NO TRADE	Low model probability, HIGH structural-break risk, high-materiality one-sided catalyst, pair invalidity, or hard implementation/risk failure.
ABSTAIN	Evidence retrieval failed, source timestamps are unreliable, agent outputs are invalid/contradictory, critical features are missing, or the system cannot make a defensible classification.

12.3 Threshold governance

The numeric thresholds above are starting configuration values, not test-set-optimised constants. Final policy thresholds are selected using training/validation results and then frozen before the final test. The UI should display the policy version alongside the model version so that results remain auditable.

13. Backtesting and evaluation framework

13.1 Evaluation ladder

Version	What changes	Question answered
Baseline 0 - threshold only	Trade every qualifying divergence threshold crossing.	What does the naive statistical-arbitrage rule do?
Baseline 1 - valid pairs only	Require rolling relationship validity before accepting the signal.	What value comes from the econometric filter alone?
Model A - ML filter	Require XGBoost score above the selected policy threshold.	Does supervised ML select better divergence events?
Model B - ML + regime/event features	Add validated contextual features.	Do additional state variables improve prediction?
Full system - ML + structural-break veto / abstention	Apply agent evidence and deterministic controller.	Does language AI reduce failed opportunities without vetoing too many successful ones?

13.2 Predictive metrics

Metric	Why it is reported
ROC-AUC	Overall ranking quality across decision thresholds.
PR-AUC / precision / recall	More informative if successful and failed events are imbalanced.
F1 score	Single threshold-based balance between precision and recall.
Brier score	Quality of the probability estimates themselves.
Calibration curve	Whether events assigned similar probabilities revert at similar observed rates.
Performance by probability bucket	Tests whether higher model scores correspond to better outcomes.

13.3 Trading and risk metrics

Metric family	Reported outputs
Per-event	Success rate, gross return, net return after costs, holding period, adverse excursion, favourable excursion.
Portfolio-level	Cumulative return, annualised return, Sharpe ratio, maximum drawdown, turnover, exposure, number of concurrent pairs.
Tail behaviour	Worst trade, lower-tail return quantiles, drawdown contribution of failed structural-break events.
Implementation sensitivity	Results under higher transaction-cost and short-borrow assumptions.

13.4 Agent-specific metrics

Metric	Definition in project terms
Failed-opportunity rejection rate	Share of baseline failures that the agent/controller correctly rejects or abstains on.
False-veto rate	Share of successful baseline opportunities incorrectly rejected by the agent/controller.
Event-classification accuracy	Agreement between extracted event type/materiality and a human-labelled evaluation subset.
Evidence validity rate	Share of agent conclusions supported by valid pre-event source evidence.
Abstention rate	Share of events for which the system explicitly declines to decide because evidence or data is inadequate.

13.5 Robustness slices

- By pair and by sector.
- By calendar year and market regime.
- By positive versus negative divergence.
- By probability bucket and break-risk category.
- By event type and materiality where labels exist.
- With and without each major feature family (ablation).
- Across multiple transaction-cost assumptions.

14. Complete user interface specification

14.1 Navigation

Page	Purpose
1. Scanner	Portfolio-level view of current/historical divergence candidates and final research states.
2. Pair Detail	Deep dive into one pair/event: relationship, ML, evidence, agents, and simulated trade state.
3. Strategy Lab	Backtest comparisons, ablations, robustness, rejected-trade analysis, and cost sensitivity.
4. Model & Data Monitor	Dataset, model, calibration, drift, source health, and processing status.
5. Methodology / Model Card	Versioned assumptions, feature schema, label rules, limitations, and reproducibility metadata.

14.2 Page 1 - Relative-Value Scanner

UI region	Specification
Top status bar	Data as-of timestamp, active model version, policy version, document-source status, last batch refresh.
KPI cards	Active divergences; CANDIDATE count; REVIEW count; HIGH break-risk count; average calibrated reversion probability.
Filters	Date/as-of selector; sector; decision status; minimum probability; break risk; pair status; catalyst type.
Main table	Pair, sector, current z-score, relationship status, half-life, calibrated reversion probability, top SHAP driver, break risk, catalyst, final status, last updated.
Visual encoding	Decision badges and break-risk badges; compact spread sparkline; warning icon for missing/stale evidence.
Interaction	Click a row to open Pair Detail for that event; export selected rows as CSV for report analysis.

14.3 Page 2 - Pair Detail

Tab / panel	Contents
Overview	Pair identity, decision badge, pair status, ML probability, break risk, current z-score, simulated long/short direction, main catalyst.
Price & spread	Normalised price chart; spread / z-score chart; divergence threshold, stop, convergence band, historical event markers.
Relationship	Current and historical cointegration diagnostics; hedge-ratio history; half-life; rolling correlation; stability flags; pair lifecycle timeline.
ML explanation	Probability gauge; top SHAP waterfall/bar; feature-state table; calibration bucket and historical success rate for similar scores.
Evidence timeline	Chronological source cards for both companies with timestamp, source type, event type, affected leg, materiality, and citation link/ID.
Agent debate	Side-by-side Mean-Reversion and Structural-Break arguments; each claim tied to evidence; Risk/Validation findings beneath.
Decision rationale	Deterministic controller gates with PASS/FAIL/UNKNOWN state; final decision and reason codes.
Simulated trade	Research entry state, hedge sizing, exit/stop rules, estimated transaction-

Tab / panel	Contents
	cost assumptions, and historical outcome when viewing a completed event.

14.4 Page 3 - Strategy Lab

Panel	Contents
System comparison	Baseline 0, Baseline 1, Model A, Model B, Full System metrics in one table.
Equity / cumulative result view	Interactive curves with gross and net-cost toggles.
Prediction quality	ROC, precision-recall, calibration curve, probability-bucket success plot.
Ablation	Remove one feature family or agent layer and compare metric deterioration.
Rejected-trade analysis	How many signals were vetoed; how many would have failed; how many successful opportunities were lost.
Robustness	Sector, pair, year, regime, divergence-direction, event-type breakdowns.
Cost sensitivity	Net results across transaction-cost and short-borrow scenarios.

14.5 Page 4 - Model & Data Monitor

Panel	Contents
Data health	Latest price date; missing values; stale pairs; failed downloads; SEC retrieval status; document counts.
Dataset health	Event count by partition, class balance, pairs represented, feature missingness, overlap checks.
Model health	Model version, training window, validation/test metrics, calibration summary, feature schema hash.
Drift	Feature-distribution shifts and change in observed mean-reversion success rate by recent period.
Agent health	Run success rate, abstention rate, source-validation failures, average latency, cache hit rate.
Job status	Last successful ingestion, feature build, model training, agent batch, and backtest run.

14.6 Page 5 - Methodology / Model Card

- Project purpose and scope boundaries.
- Current pair-universe definition and data period.
- Event and label policy.
- Feature families and excluded feature classes.
- Model and calibration versions.
- Agent roles, evidence rules, and abstention policy.
- Backtest transaction-cost assumptions.
- Known limitations and prototype-versus-deployment disclaimer.
- Reproduction identifiers: dataset version, Git commit, model artefact hash, prompt/policy versions.

15. Exact technical stack and engineering architecture

15.1 Chosen stack

Layer	Technology	Purpose
Language / runtime	Python 3.11	Single language for data, econometrics, ML, agents, backtesting, and UI.
Environment / packaging	uv + pyproject.toml	Reproducible dependency management and fast local setup.
Tabular processing	pandas + NumPy	Price cleaning, feature construction, event tables, backtest calculations.
Analytics database	DuckDB	Local analytical queries across pair snapshots, events, predictions, agent findings, and trades.
Columnar files	Parquet	Versioned raw/processed tables with efficient local storage.
Econometrics	statsmodels	OLS relationship estimation, Engle-Granger cointegration, residual diagnostics, time-series utilities.
Classical ML	scikit-learn	Baselines, metrics, calibration, preprocessing, evaluation utilities.
Primary ML	XGBoost	Main binary event classifier.
Explainability	SHAP	Local and global feature attribution.
Optional regime model	hmmlearn	Hidden Markov Model regime feature if included.
Schema / validation	Pydantic + Pandera	Typed agent state, structured outputs, dataframe/data-contract checks.
LLM abstraction	LiteLLM	Provider-agnostic model calls so the project is not locked to one vendor.
Agent orchestration	LangGraph	Deterministic state graph for retrieval, extraction, adversarial analysis, validation, retries, and abstention.
UI	Streamlit	Multi-page research application and interaction layer.
Charts	Plotly	Interactive price, spread, calibration, equity, and diagnostics visualisations.
Experiment tracking	MLflow (local/file-backed)	Model runs, parameters, metrics, artefact registration.
Model serialisation	joblib / native XGBoost save format	Versioned reproducible inference artefacts.
Testing	pytest	Unit, integration, leakage, and regression tests.
Code quality	Ruff + mypy	Formatting/linting and type checking.
Version control	Git + GitHub	Collaboration, review, tagged releases, issue tracking.
CI	GitHub Actions	Run tests/lint on pull requests and main branch.
Container	Docker	Reproducible local/demo environment if needed.
Deployment	Streamlit Community Cloud or equivalent course-approved free tier	Public demo with secrets stored outside the repository.

15.2 Architectural choice: no separate API for the course build

The Streamlit app imports the project service modules directly. A separate FastAPI layer is not required for the course submission because it would duplicate interfaces without improving the research result. Internal boundaries are still explicit: data services, econometrics, features, ML, agents, controller, backtester, and UI remain separate Python packages. A REST API is a production extension, not an MVP requirement.

15.3 Configuration strategy

- All key research parameters live in version-controlled YAML/TOML configuration files rather than being scattered through notebooks or UI code.
- Secrets such as LLM API keys are provided through environment variables or Streamlit secrets and never committed to Git.
- Model, data, prompt, policy, and pair-universe versions are recorded on every stored prediction and agent run.
- Historical outputs are cached so the live demo does not depend on external APIs being available during presentation.

16. Data model, repository structure, and system jobs

16.1 Core analytical tables

Table / entity	Key contents
pairs	pair_id, tickers, company names, sector/sub-industry, lifecycle status, start/end eligibility, notes.
prices	ticker, date, adjusted OHLCV, source, ingestion timestamp, dataset version.
pair_snapshots	pair_id, as_of_date, hedge ratio, cointegration diagnostics, half-life, correlation, spread state, pair status.
divergence_events	event_id, pair_id, event timestamp, divergence direction, entry z-score, label horizon, outcome status.
event_features	event_id plus full versioned feature vector and missingness flags.
documents	document_id, company/ticker, filing/news type, publication timestamp, source ID/URL metadata, local content path.
event_extractions	event_id, document references, event type, affected leg, materiality, relevance, confidence, provenance.
model_predictions	event_id, model version, raw score, calibrated probability, probability bucket, SHAP summary.
agent_runs	event_id, prompt/model version, agent role, structured output, validation result, latency, source references.
decisions	event_id, policy version, gate states, final status, reason codes.
backtest_trades	event_id, simulated entry/exit, outcome type, gross/net return, cost assumptions, holding period.
experiment_runs	dataset/model/policy versions plus metrics and timestamps.

16.2 Repository layout

Path	Responsibility
app/	Streamlit entry point and page modules only.
src/config/	Typed configuration, constants, version identifiers.
src/data/	Market/document ingestion, calendar alignment, storage access, data contracts.
src/pairs/	Pair universe and lifecycle management.
src/econometrics/	Relationship estimation, diagnostics, spread state, event triggers.
src/features/	Point-in-time feature construction and feature schema.
src/ml/	Training, calibration, inference, SHAP, model artefact loading.
src/nlp/	Document cleaning, chunking, retrieval, event extraction.
src/agents/	LangGraph state, specialised agents, tool interfaces, output validation.
src/decision/	Deterministic risk gates and final status policy.
src/backtest/	Historical event execution, costs, metrics, ablation runs.

Path	Responsibility
src/ui/	Reusable tables, badges, plots, formatting helpers.
tests/	Unit, integration, leakage, and regression tests.
data/	Raw/processed/cache directories excluded from Git where appropriate.
artifacts/	Versioned models, calibration artefacts, SHAP background, model cards.
configs/	Pair universe, run configs, thresholds, policy definitions.
docs/	Data dictionary, architecture notes, methodology/model card, demo runbook.

16.3 Main system jobs

Job	Input -> output
ingest_market_data	Ticker universe -> versioned daily market dataset.
ingest_documents	Ticker/CIK list + date range -> timestamped filing/news store.
build_pair_snapshots	Prices + pair config -> rolling relationship diagnostics.
generate_events	Pair snapshots -> non-overlapping divergence events.
build_features_and_labels	Events + point-in-time data -> supervised dataset.
train_model	Training/validation dataset -> model + calibrator + metrics + SHAP reference.
run_agent_batch	Events + admissible documents -> structured event evidence and break-risk outputs.
run_backtest	Events + model/agent/policy outputs -> baseline and full-system results.
refresh_dashboard_cache	Latest analytics tables -> demo-ready scanner and pair-detail cache.

17. Reproducibility, testing, governance, and monitoring

17.1 Required automated tests

Test family	Examples
Data-contract tests	Expected columns/types, unique IDs, no duplicate event IDs, monotonic timestamps, missingness thresholds.
Point-in-time / leakage tests	No source timestamp after event time; no rolling feature uses future dates; partition embargo respected.
Econometric tests	Known synthetic/fixture pair produces expected relationship state; invalid pair is rejected; monthly refit boundaries are correct.
ML tests	Training/inference feature order identical; model loads; calibration returns valid probabilities; SHAP calculation succeeds.
Agent tests	Invalid JSON rejected; uncited material claim fails validation; missing evidence leads to UNKNOWN/ABSTAIN; deterministic state transitions.
Backtest tests	No overlapping same-pair entries; costs deducted once; stop/exit ordering deterministic; event outcomes reproducible.
UI smoke tests	Core pages load from cached fixtures without external API access.

17.2 Reproducibility identifiers

- Git commit hash.
- Raw/processed dataset version.
- Pair-universe version.
- Feature-schema version.
- Model and calibration version.
- Prompt/agent graph version.
- Decision-policy version.
- Backtest configuration version.

17.3 Governance controls

Risk	Application control
Data snooping / multiple testing	Curated economic pair universe; limited pair search; final test frozen; full experiment log.
Look-ahead bias	Point-in-time feature store, source timestamp checks, chronological partitions, label embargo.
Structural break	Rolling re-estimation, pair retirement, ML filter, event investigation, abstention.
LLM hallucination	Bounded retrieval, structured outputs, source provenance, validator agent, deterministic controller.
Model drift	Feature and outcome monitoring, recent-period evaluation, re-training/revalidation trigger.
Backtest optimism	Explicit costs, sensitivity analysis, survivor-bias disclosure, no live-alpha claim.
External API failure	Local cache, retry/timeout policies, demo mode using precomputed historical results.

17.4 Monitoring in the app

The monitor page is part of the product rather than an afterthought. It exposes stale data, failed source jobs, model version, calibration state, feature drift, recent performance, agent failure rate, and abstention rate. This makes it possible to demonstrate not only model accuracy but also operational control.

18. Prototype versus real deployment

Area	Course prototype demonstrates	Real deployment would additionally require
Market data	Daily public adjusted OHLCV and public market series.	Licensed, survivorship-bias-free security master; delistings; corporate actions; robust historical reference data; live feeds.
Pair universe	Curated long-lived large-cap peers.	Point-in-time universe membership, automated lifecycle governance, corporate-event handling, capacity/liquidity constraints.
Execution	Research long/short simulation with configurable costs.	Borrow availability and live borrow fees, bid/ask, slippage, market impact, order management, execution controls, broker integration.
Text evidence	SEC filings plus optional public news.	Licensed low-latency news, complete historical archives, entitlements, legal review of alternative data, source redundancy.
ML	Chronological out-of-sample classification and calibration.	Continuous monitoring, formal retraining triggers, champion/challenger models, independent validation, incident management.
Agents	Source-grounded research investigation with abstention.	Enterprise retrieval, access controls, audit logging, model/vendor governance, adversarial testing, cost/latency SLAs.
Risk	Simple portfolio/event controls and sensitivity analysis.	Capital allocation, leverage/margin, factor exposure, stress testing, concentration limits, kill switches, compliance surveillance.
Operations	Single research app and scheduled jobs.	Highly available services, observability, secret management, disaster recovery, data lineage, change approval, production support.

Claim boundary

A successful course result is evidence that the AI layers improve historical event selection and structural-break risk control relative to defined baselines. It is not evidence that the strategy is ready for live capital or that the reported historical returns are achievable in production.

19. Build sequence and acceptance criteria

19.1 Recommended implementation order

1. Freeze pair universe, sample dates, event/label rule, and train/validation/test partitions.
2. Build market-data ingestion, DuckDB/Parquet storage, pair snapshots, and leakage tests.
3. Generate the historical divergence-event dataset and baseline backtest.
4. Build feature pipeline, logistic benchmark, XGBoost model, calibration, and SHAP.
5. Build Scanner and Pair Detail using only quantitative outputs so the app is already functional.
6. Add SEC document ingestion, retrieval, structured event extraction, and human-labelled validation subset.
7. Add adversarial agents, validator, deterministic controller, and caching.
8. Build Strategy Lab and full ablation evaluation.
9. Add Model & Data Monitor, model card, demo mode, deployment, and presentation polish.
10. Only after the above is stable: optional HMM, broader news, LLM event features inside XGBoost, or other stretch work.

19.2 Functional acceptance criteria

Area	Minimum acceptance condition
Data	A fresh environment can rebuild the processed dataset from documented sources or use a frozen reproducible project snapshot.
Econometrics	Pair status and divergence events are generated point-in-time with automated tests.
ML	Model trains on chronological data, is calibrated, saves/loads correctly, and

Area	Minimum acceptance condition
	beats the transparent baseline on at least one pre-specified predictive metric without using the final test for tuning.
Agents	Every material conclusion is source-grounded; failed evidence retrieval produces abstention; structured outputs validate reliably.
Evaluation	All baseline/ablation variants run on the same event set and cost assumptions; final test results are reproducible.
UI	Scanner -> Pair Detail -> Strategy Lab flow works end-to-end from cached project data during the demo.
Governance	Model/data/prompt/policy versions are visible; limitations and deployment boundary are explicit.

Appendix A. Working configuration defaults

Parameter	Default	Notes
Asset market	US large-cap equities	Chosen primarily for reliable SEC filing access and long daily histories.
Pair count	20-40	Manually curated same-sector / same-sub-industry pairs.
Raw data period	2016-01-01 to 2025-12-31	Creates a stable frozen research sample.
Formation window	504 trading days	Rolling pair relationship estimation.
Relationship refit	Monthly	Point-in-time historical reconstruction.
Cointegration threshold	5% starting rule	Final value fixed using methodology policy before final test.
Divergence trigger	|z| >= 2.0	Starting event threshold selected to provide enough events for supervised learning.
Convergence band	|z| <= 0.5	Success target.
Adverse stop	|z| >= 3.0 in the adverse direction	Failure control.
Maximum label horizon	10 trading days	Event resolution horizon.
Train	2018-2021 usable events	After sufficient formation history.
Validation	2022-2023	Threshold/model/policy selection and calibration.
Final test	2024-2025	Untouched until methodology is frozen.
Primary classifier	XGBoost	Main tabular model.
Probability calibration	Sigmoid / Platt	Validation-based calibration.
Candidate probability threshold	0.65 starting value	Validation-tuned and then frozen.
Canonical historical documents	SEC EDGAR	Point-in-time evidence source.
Agent evidence lookback	10 calendar days plus most recent relevant filing context	All documents must pre-date the event timestamp.
Transaction cost	10 bps per leg round-trip starting case	Strategy Lab also reports higher-cost scenarios.
Decision statuses	CANDIDATE / REVIEW / NO TRADE / ABSTAIN	Controller output only.

Parameter discipline

These defaults are the working project specification, not post-hoc choices. If the team changes them, the change should be documented before final-test evaluation and the experiment/policy version should change.

Appendix B. App-specific concept glossary

Finance and econometrics

Concept	Role inside this app
Relative value	The project trades the difference between related securities rather than making a stand-alone directional forecast.
Long / short pair	Simulated research position: long the relatively cheap leg and short the relatively rich leg under the estimated relationship.
Hedge ratio	Relative sizing relationship estimated from historical prices and used to construct the spread and simulated pair exposure.
Spread	Residual relative-value state of the pair after applying the estimated relationship.
Z-score	Standardised measure of how unusual the current spread is relative to its recent history; used to trigger divergence events.
Cointegration	Rolling statistical validity test used to decide whether a pair is eligible for the opportunity detector.
Stationarity / residual stability	Property required for the spread to behave like a mean-reverting relationship rather than an unconstrained drift.
Mean reversion	Successful event outcome when the spread moves back into the pre-defined convergence zone.
Half-life	Estimated speed of spread reversion; used as a feature and diagnostic.
Structural break	Persistent change in the economic or statistical relationship that can make historical pair behaviour misleading.
Rolling window	Historical estimation window that moves through time so every pair state is reconstructed using only information then available.
Point-in-time estimation	Historical reconstruction discipline that prevents future data from entering a past signal.
Transaction costs	Explicit deduction for trading the two legs; varied in robustness analysis.
Short borrow	Cost/availability risk of the short leg; simplified in the prototype and a major live-deployment requirement.
Sharpe ratio	Risk-adjusted performance metric reported for the simulated strategy comparison.
Maximum drawdown	Largest peak-to-trough loss in the simulated portfolio path.
Turnover	Amount of position change generated by the strategy and a driver of implementation cost.

AI, ML, and NLP

Concept	Role inside this app
Supervised binary classification	Learns from labelled historical divergence events to estimate whether a new event will satisfy the mean-reversion success rule.
Gradient boosting / XGBoost	Primary predictive model for structured event features.
Probability calibration	Makes the displayed model probability more interpretable as an empirical success likelihood.
Feature engineering	Transforms market, relationship, security, liquidity, and event state into model-ready predictors.
SHAP	Explains which features increased or decreased the current model score.
HMM regime state	Optional latent market-state feature used to represent changing market conditions.

Concept	Role inside this app
NLP	Processing and classification of filings/news into structured event evidence.
Retrieval	Selects a small relevant evidence set from stored documents before an LLM is asked to analyse it.
Structured output	Machine-readable LLM result constrained to a fixed schema rather than unconstrained prose.
Agent	Specialised LLM-powered component with a bounded task, tools, state, and output contract.
Adversarial agent debate	Separate agents construct competing mean-reversion and structural-break explanations from the same evidence.
Abstention	Explicit system state used when evidence or data is insufficient for a defensible conclusion.
Hallucination control	Source-bounded retrieval, structured schemas, provenance, validation, and deterministic decision logic.

Engineering and coding architecture

Concept	Role inside this app
ETL	Ingests, cleans, aligns, and versions market and document data before modelling.
Data contract	Schema rules that ensure every job receives and returns expected columns, types, IDs, and timestamps.
DuckDB / Parquet	Local analytical storage pattern for reproducible project-scale data.
Artefact	Saved model, calibrator, feature schema, SHAP reference, or report output tied to a versioned experiment.
Caching	Stores expensive historical computations and agent outputs so the UI remains fast and demo-safe.
Versioning	Tracks data, model, prompt, policy, and code versions so a displayed result can be reproduced.
Unit test	Tests one deterministic function or module in isolation.
Integration test	Tests that multiple layers such as event -> features -> model -> controller work together.
Leakage test	Automated check that no feature/source timestamp violates the event information set.
CI	GitHub automation that runs tests and code-quality checks before changes are merged.
Model card	Versioned methodology and limitation record presented in the app.
Demo mode	Uses cached, precomputed outputs so the presentation remains functional if external data or LLM APIs fail.

End-state product

A completed system should let a user move from a flagged pair -> quantitative relationship diagnostics -> calibrated ML score -> source-grounded structural-break evidence -> controlled research decision -> transparent historical evaluation, while preserving a complete audit trail across every layer.